

ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC BÁCH KHOA  
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



## Cấu Trúc Rời Rạc

Báo cáo Bài Tập Lớn 252

Giảng viên lý thuyết: Trần Hồng Tài  
SV thực hiện: Nguyễn Cảnh Nguyên – 2312357

TP Hồ Chí Minh, Tháng 5 Năm 2026

# Contents

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>Tổng quan thuật toán A*</b>                                     | <b>2</b>  |
| <b>1 Task 1 – Degrees of Separation in a Social Network</b>        | <b>4</b>  |
| 1.1 Mô tả bài toán . . . . .                                       | 4         |
| 1.2 Heuristic: BFS ngược từ Goal . . . . .                         | 4         |
| 1.3 Manual Run . . . . .                                           | 4         |
| 1.4 Code hiện thực hàm chính . . . . .                             | 5         |
| <b>2 Task 2 – Drone Delivery in 2D Space with Three Heuristics</b> | <b>7</b>  |
| 2.1 Mô tả bài toán . . . . .                                       | 7         |
| 2.2 Ba loại Heuristic . . . . .                                    | 7         |
| 2.3 So sánh ba loại Heuristic qua các ví dụ . . . . .              | 7         |
| 2.4 Code hiện thực . . . . .                                       | 8         |
| <b>3 Task 3 – Warehouse Robot Navigation with Obstacles</b>        | <b>10</b> |
| 3.1 Mô tả bài toán . . . . .                                       | 10        |
| 3.2 Vì sao Manhattan không admissible ở bài này? . . . . .         | 10        |
| 3.3 Manual Run test case 9 task 3 . . . . .                        | 10        |
| 3.4 So sánh hai mode . . . . .                                     | 11        |
| 3.5 Code hiện thực . . . . .                                       | 11        |
| <b>4 Task 4 – Evacuation Route Planning</b>                        | <b>14</b> |
| 4.1 Mô tả bài toán . . . . .                                       | 14        |
| 4.2 Chuyển đổi từ Grid sang Graph (Adjacency Matrix) . . . . .     | 14        |
| 4.3 Chạy tay ví dụ so sánh hai mode . . . . .                      | 14        |
| 4.4 So sánh hai mode . . . . .                                     | 16        |
| 4.5 Code hiện thực . . . . .                                       | 16        |

## Tổng quan thuật toán A\*

Thuật toán A\* (A-star) là một thuật toán tìm đường đi ngắn nhất trên đồ thị có trọng số hoặc bản đồ dạng lưới. A\* kết hợp sức mạnh của thuật toán Dijkstra và Greedy Best-First Search. Tại mỗi bước, A\* chọn node có giá trị  $f(n)$  nhỏ nhất để mở rộng:

$$f(n) = g(n) + h(n)$$

Trong đó:

- $g(n)$ : chi phí thực tế từ Start đến node  $n$
- $h(n)$ : ước lượng (heuristic) chi phí từ  $n$  đến Goal
- $f(n)$ : tổng chi phí ước tính của đường đi qua  $n$

**Điều kiện để A\* tìm ra đường tối ưu:** Hàm  $h(n)$  phải là *admissible*, tức là không bao giờ đánh giá cao hơn chi phí thực tế ( $h(n) \leq h^*(n)$ ).

**Các cấu trúc dữ liệu chính:**

- **MinHeap:** hàng đợi ưu tiên lưu các node theo  $f$  tăng dần, tie-break theo  $h$  nhỏ hơn trước. (Data Structure này là tự hiện thực thêm với <vector> không import thư viện ngoài)
- **visited[]:** mảng đánh dấu Closed Set
- **parents[]:** mảng lưu node cha để truy vết đường đi ngược từ Goal về Start
- **PathNode:** linked list chứa kết quả, mỗi node lưu **name**,  $f$ ,  $g$ ,  $h$

## Hiện thực helper functions

Assignment chỉ yêu cầu hiện thực các hàm `findSocialPath`, `findDronePath`, `findWarehousePath`, `findEvacuationPath` và cấu trúc `PathNode` (đã cho sẵn tên field). Hiện thực thêm helper để tiện cho việc code gồm:

- **Constructor cho PathNode (PathNode.h):** Assignment cho sẵn struct với 4 field (**name**, **f**, **g**, **h**, **next**) nhưng không có constructor. Em thêm constructor ở PathNode:

```
1 struct PathNode {
2     string name;
3     double f;
4     double g;
5     double h;
6     PathNode* next;
7     //bonus
8     PathNode(string name, double f, double g, double h)
9         : name(name), f(f), g(g), h(h), next(nullptr) {}
10 };
```

- **Struct MinHeap (Algo.h):** Assignment không cho sử dụng thư viện ngoài nên em tự hiện thực min-heap với các thao tác `push()` và `pop()`, xét dựa trên  $f$  và  $h$ .

```
1 struct HeapNode {
2     double f;
3     int id;
4     double h;
5 };
6 struct MinHeap {
7     vector<HeapNode> data;
8     bool empty() const { return data.empty(); }
9     void push(double f, double h, int id) {
10         data.push_back({f, id, h});
11     }
```

```

11         int i = data.size() - 1;
12         while (i > 0) { // heapify-up
13             int father = (i - 1) / 2;
14             if (data[father].f < data[i].f ||
15                 (data[father].f == data[i].f && data[father].h <= data[
16                     i].h)) break;
17             swap(data[father], data[i]);
18             i = father;
19         }
20     HeapNode pop() {
21         HeapNode ncn = data[0];
22         data[0] = data.back(); data.pop_back();
23         int i = 0, size = data.size();
24         while (true) { // heapify-down
25             int l = 2*i + 1, r = 2*i + 2, temp = i;
26             if (l < size && (data[l].f < data[temp].f ||
27                 (data[l].f == data[temp].f && data[l].h <= data[temp].h
28                     ))) temp = l;
29             if (r < size && (data[r].f < data[temp].f ||
30                 (data[r].f == data[temp].f && data[r].h <= data[temp].h
31                     ))) temp = r;
32             if (temp == i) break;
33             swap(data[temp], data[i]);
34             i = temp;
35         }
36     };

```

- **Hàm build\_linked\_list** (Algo.cpp): Hàm helper dùng chung cho cả 4 task, nhận vào vector index của đường đi và xây dựng linked list PathNode từ đầu đến cuối.

```

1 PathNode* build_linked_list(const vector<int>& path, const double g[],
2                             const double h[], const string names[]) {
3     PathNode* head = nullptr;
4     PathNode* tail = nullptr;
5     for (int idx : path) {
6         PathNode* node = new PathNode(names[idx], g[idx] + h[idx], g[
7             idx], h[idx]);
8         if (!head) head = tail = node;
9         else { tail->next = node; tail = node; }
10    }
11    return head;

```

# 1 Task 1 – Degrees of Separation in a Social Network

## 1.1 Mô tả bài toán

Cho một đồ thị mạng xã hội được biểu diễn bởi ma trận kề ( $\text{adjMatrix}[i][j] = 1$  nghĩa là có kết nối giữa người  $i$  và người  $j$ ). Triển khai thuật toán  $A^*$  để tìm đường đi có ít kết nối nhất từ  $\text{startPerson}$  đến  $\text{goalPerson}$ .

**Input:** Ma trận kề, người bắt đầu, người đích.

**Output:** Một linked list `PathNode` chứa đường đi từ start đến goal, mỗi node lưu tên người (index),  $f(n)$ ,  $g(n)$ ,  $h(n)$ .

**Hàm chi phí:**

- $g(n)$ : số kết nối đã đi từ start đến node hiện tại
- $h(n)$ : ước lượng số kết nối tối thiểu còn lại để đến goal
- $f(n) = g(n) + h(n)$

## 1.2 Heuristic: BFS ngược từ Goal

Vì tất cả cạnh có trọng số bằng 1, ta có thể tính chính xác khoảng cách ngắn nhất từ mọi node đến Goal bằng BFS ngược:

1. Khởi tạo  $h(i) = +\infty$  cho tất cả node, đặt  $h(\text{goal}) = 0$ .
2. Đưa goal vào queue.
3. Với mỗi node  $u$  lấy ra từ queue, duyệt tất cả  $i$  có cạnh  $i \rightarrow u$ .
4. Nếu  $h(u) + 1 < h(i)$ , cập nhật  $h(i)$  và đưa  $i$  vào queue.

Vì  $h(n)$  tính chính xác khoảng cách thực tế, nó luôn thỏa mãn admissible.  $A^*$  với heuristic này sẽ chỉ mở đúng các node trên đường đi ngắn nhất.

Listing 1: Hàm heuristic\_task\_1 trong Algo.cpp

```
1 void heuristic_task_1(int goal, double matrix[100][100], double h[100]) {
2     vector<int> q;
3     q.push_back(goal);
4     h[goal] = 0.0;
5     int head = 0;
6     while (head < q.size()) {
7         int u = q[head];
8         for (int i = 0; i < 100; i++) {
9             if (matrix[i][u] == 1.0) { // co canh i -> u
10                 if (h[u] + 1 < h[i]) {
11                     h[i] = h[u] + 1;
12                     q.push_back(i);
13                 }
14             }
15         }
16         head++;
17     }
18 }
```

## 1.3 Manual Run

Đồ thị 5 node (vô hướng):

$$\text{adjMatrix} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Start = 0, Goal = 4.

Bước 1: Tính  $h(n)$  bằng BFS ngược từ goal = 4.

| BFS step | Node | Cập nhật               | Mảng h                                |
|----------|------|------------------------|---------------------------------------|
| Init     | –    | $h(4) = 0$             | $[\infty, \infty, \infty, \infty, 0]$ |
| 1        | 4    | $h(3) = 1$             | $[\infty, \infty, \infty, 1, 0]$      |
| 2        | 3    | $h(0) = 2, h(2) = 2$   | $[2, \infty, 2, 1, 0]$                |
| 3        | 0    | $h(1) = 3$             | $[2, 3, 2, 1, 0]$                     |
| 4        | 2    | $h(1) \leq 3$ , bỏ qua | –                                     |

Table 1: BFS ngược tính  $h(n)$ , Goal = 4

Bước 2: Chạy A\* từ Start = 0 đến Goal = 4.

Khởi tạo:  $g(0) = 0, h(0) = 2, f(0) = 2$ . Push node 0 vào heap.

| Bước | Pop | $f$ | $g$ | $h$ | Hành động                                   |
|------|-----|-----|-----|-----|---------------------------------------------|
| 1    | 0   | 2   | 0   | 2   | Push node 1 ( $f = 4$ ), node 3 ( $f = 2$ ) |
| 2    | 3   | 2   | 1   | 1   | Push node 2 ( $f = 4$ ), node 4 ( $f = 2$ ) |
| 3    | 4   | 2   | 2   | 0   | <b>Goal!</b> Dừng.                          |

Table 2: A\* trace Task 1, Start=0, Goal=4

Truy vết:  $4 \leftarrow 3 \leftarrow 0$ . Đường đi:  $0 \rightarrow 3 \rightarrow 4$ .

Output:

```
Node: 0 | f: 2 | g: 0 | h: 2
Node: 3 | f: 2 | g: 1 | h: 1
Node: 4 | f: 2 | g: 2 | h: 0
```

## 1.4 Code hiện thực hàm chính

Listing 2: Hàm findSocialPath trong Algo.cpp

```
1 PathNode *findSocialPath(double adjMatrix[100][100], int startPerson,
2                           int goalPerson) {
3     double g[100], h[100];
4     int parent[100];
5     string names[100];
6     bool visited[100];
7     vector<int> path;
8     MinHeap mcn;
9     if (startPerson == goalPerson) {
10         PathNode *head = nullptr;
11         string name = to_string(startPerson);
12         head = new PathNode(name, 0, 0, 0);
13         return head;
14     };
15
16     for (int i = 0; i < 100; ++i) {
```

```

17     g[i] = MAX;
18     h[i] = MAX;
19     names[i] = to_string(i);
20     visited[i] = false;
21     parent[i] = -1;
22 };
23 heuristic_task_1(goalPerson, adjMatrix, h);
24
25 g[startPerson] = 0;
26 ncn.push(g[startPerson] + h[startPerson], h[startPerson], startPerson);
27 while (!ncn.empty()) {
28     HeapNode temp = ncn.pop();
29     if (visited[temp.id] == true)
30         continue;
31     if (temp.id == goalPerson) {
32         break;
33     };
34
35     visited[temp.id] = true;
36
37     for (int i = 0; i < 100; ++i) {
38         if (adjMatrix[temp.id][i] == 1.0) {
39             if (g[temp.id] + 1 < g[i]) {
40                 g[i] = g[temp.id] + 1;
41                 parent[i] = temp.id;
42                 ncn.push(g[i] + h[i], h[i], i);
43             };
44         };
45     };
46 };
47
48 if (parent[goalPerson] == -1)
49     return nullptr;
50 int i = goalPerson;
51 while (parent[i] != -1) {
52     path.push_back(parent[i]);
53     i = parent[i];
54 };
55 reverse(path.begin(), path.end());
56 path.push_back(goalPerson);
57 return build_linked_list(path, g, h, names);
58 };

```

## 2 Task 2 – Drone Delivery in 2D Space with Three Heuristics

### 2.1 Mô tả bài toán

Cho một đồ thị có trọng số biểu diễn không gian giao hàng 2D. Mỗi node là một điểm giao hàng với tọa độ  $\text{coords}[i] = \{x, y\}$ . Ma trận kề  $\text{adjMatrix}[i][j]$  lưu khoảng cách bay giữa hai điểm (0 = không có đường bay trực tiếp). Tìm đường bay có tổng chi phí nhỏ nhất từ  $\text{startPoint}$  đến  $\text{goalPoint}$ .

**Input:** Ma trận kề có trọng số, mảng tọa độ 2D, điểm xuất phát, điểm đích, mode heuristic.

**Output:** Linked list  $\text{PathNode}$  chứa đường đi, mỗi node lưu tọa độ dạng " $(x,y)$ ",  $f(n)$ ,  $g(n)$ ,  $h(n)$ .

**Hàm chi phí:**

- $g(n)$ : tổng trọng số các cạnh đã đi từ điểm xuất phát
- $h(n)$ : khoảng cách ước lượng từ node hiện tại đến đích theo heuristic được chọn
- $f(n) = g(n) + h(n)$

### 2.2 Ba loại Heuristic

| Mode | Tên       | Công thức                                  | Ứng dụng            |
|------|-----------|--------------------------------------------|---------------------|
| 1    | Manhattan | $h =  x_1 - x_2  +  y_1 - y_2 $            | Di chuyển theo lưới |
| 2    | Euclidean | $h = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ | Di chuyển tự do     |
| 3    | Chebyshev | $h = \max( x_1 - x_2 ,  y_1 - y_2 )$       | Di chuyển 8 hướng   |

Table 3: Ba heuristic mode cho Task 2

Quan hệ:  $\text{Chebyshev} \leq \text{Euclidean} \leq \text{Manhattan}$  với mọi cặp điểm.

### 2.3 So sánh ba loại Heuristic qua các ví dụ

Mỗi heuristic sẽ phát huy thế mạnh (ước lượng sát nhất khoảng cách thực tế, từ đó mở rộng ít node nhất) tùy thuộc vào cấu trúc kết nối, hướng di chuyển cho phép và trọng số cạnh của đồ thị giao hàng.

**Ví dụ 1: Lưới đường phố (Manhattan)**

Các điểm giao hàng nằm trên một lưới vuông, drone chỉ được phép bay theo các trục ngang/dọc. Khoảng cách giữa các điểm kề nhau là 1.

- **Nodes:**  $A(0,0)$  [Start],  $B(1,0)$ ,  $C(0,1)$ ,  $D(1,1)$  [Goal].
- **Cạnh (cost=1):**  $A - B$ ,  $A - C$ ,  $B - D$ ,  $C - D$  (không có đường chéo).

Chi phí thực tế của đường đi ngắn nhất  $A \rightarrow D$  là 2.

- $h_{\text{Manhattan}}(A) = |1 - 0| + |1 - 0| = 2$ .
- $h_{\text{Euclidean}}(A) = \sqrt{1^2 + 1^2} \approx 1.41$ .
- $h_{\text{Chebyshev}}(A) = \max(1, 1) = 1$ .

**Nhận xét:** Manhattan ước lượng hoàn hảo khoảng cách thực tế ( $h = h^* = 2$ ), giúp A\* chỉ tập trung mở rộng các node nằm chính xác trên đường đi ngắn nhất mà không bị phân tán. Euclidean và Chebyshev đánh giá thấp hơn chi phí thực (underestimate), khiến A\* phải xét nhiều node nhánh dư thừa hơn ở các đồ thị lớn. Do đó, Manhattan có lợi thế tuyệt đối trên lưới di chuyển 4 hướng.

**Ví dụ 2: Lưới 8 hướng có chi phí đồng đều (Chebyshev)**

Drone có thể bay theo 8 hướng (thêm 4 hướng chéo), và chi phí bay chéo bằng chi phí bay thẳng (cost=1).

- **Nodes:**  $A(0, 0)$  [Start],  $B(1, 1)$  [Goal].
- **Cạnh (cost=1):**  $A \rightarrow B$  (bay chéo trực tiếp).

Chi phí thực tế  $A \rightarrow B$  là 1.

- $h_{\text{Chebyshev}}(A) = \max(1, 1) = 1.$
- $h_{\text{Euclidean}}(A) = \sqrt{1^2 + 1^2} \approx 1.41.$
- $h_{\text{Manhattan}}(A) = 1 + 1 = 2.$

**Nhận xét:** Chebyshev ước lượng **hoàn hảo** ( $h = h^* = 1$ ). Ngược lại, Manhattan và Euclidean ước lượng cao hơn chi phí thực tế (overestimate). Khi  $h(n) > h^*(n)$ , heuristic trở thành inadmissible và thuật toán A\* không còn đảm bảo tìm được đường đi tối ưu. Do đó, Chebyshev là lựa chọn tối ưu và an toàn nhất cho di chuyển 8 hướng chi phí đều (= 1).

### Ví dụ 3: Không gian bay tự do (Euclidean)

Drone bay tự do theo đường chim bay giữa các điểm bất kỳ. Trọng số cạnh chính là khoảng cách đường thẳng và chi phí đường chéo là lớn hơn đường thẳng (= 1.5) (khoảng cách Euclidean).

- **Nodes:**  $A(0, 0)$  [Start],  $B(3, 4)$  [Goal],  $C(3, 0)$ .
- **Cạnh:**  $A \rightarrow B$  (cost=5),  $A \rightarrow C$  (cost=3),  $C \rightarrow B$  (cost=4).

Chi phí thực  $A \rightarrow B$  (bay trực tiếp) là 5.

- $h_{\text{Euclidean}}(A) = \sqrt{3^2 + 4^2} = 5.$
- $h_{\text{Manhattan}}(A) = 3 + 4 = 7.$
- $h_{\text{Chebyshev}}(A) = \max(3, 4) = 4.$

**Nhận xét:** Euclidean phản ánh chính xác tuyệt đối bản chất vật lý của đồ thị không gian. Manhattan đánh giá quá cao ( $7 > 5$ ) nên inadmissible, có thể khiến A\* chọn nhầm đường vòng nếu có nhiều vật cản. Chebyshev đánh giá quá thấp ( $4 < 5$ ) nên dù admissible nhưng tính định hướng về đích kém hơn Euclidean (mở rộng nhiều node hơn mức cần thiết). Euclidean là lựa chọn chuẩn xác nhất cho không gian 2D bay tự do.

## 2.4 Code hiện thực

Listing 3: Hàm heuristic\_task\_2 và findDronePath

```

1 double heuristic_task_2(int x1, int x2, int y1, int y2, int mode) {
2     if (mode == 1) {
3         return abs(x1 - x2) + abs(y1 - y2);
4     };
5     if (mode == 2) {
6         return sqrt(pow((x1 - x2), 2) + pow((y1 - y2), 2));
7     };
8     if (mode == 3) {
9         return max(abs(x1 - x2), abs(y1 - y2));
10    };
11    return 0.0;
12 };
13
14 PathNode *findDronePath(double adjMatrix[100][100], int coords[100][2],
15                        int startPoint, int goalPoint, int mode) {
16     double g[100], h[100];
17     bool visited[100];
18     MinHeap ncn;
19     string names[100];
20     int parents[100];
21     vector<int> path;
```

```

22     if (startPoint == goalPoint) {
23         PathNode *head = nullptr;
24         string name = "(" + to_string(coords[startPoint][0]) + "," +
25             to_string(coords[startPoint][1]) + ")";
26         head = new PathNode(name, 0, 0, 0);
27         return head;
28     }
29     for (int i = 0; i < 100; i++) {
30         g[i] = MAX;
31         h[i] = MAX;
32         visited[i] = false;
33         parents[i] = -1;
34         names[i] =
35             "(" + to_string(coords[i][0]) + "," + to_string(coords[i][1]) + ")"
36         ;
37     };
38     g[startPoint] = 0;
39     h[startPoint] =
40         heuristic_task_2(coords[startPoint][0], coords[goalPoint][0],
41             coords[startPoint][1], coords[goalPoint][1], mode);
42
43     ncn.push(g[startPoint] + h[startPoint], h[startPoint],
44         startPoint);
45     while (!ncn.empty()) {
46         HeapNode temp = ncn.pop();
47         if (temp.id == goalPoint)
48             break;
49         if (visited[temp.id] == true)
50             continue;
51         visited[temp.id] = true;
52         for (int i = 0; i < 100; ++i) {
53             if (adjMatrix[temp.id][i] != 0.0) {
54                 if (g[temp.id] + adjMatrix[temp.id][i] < g[i]) {
55                     g[i] = g[temp.id] + adjMatrix[temp.id][i];
56                     parents[i] = temp.id;
57                     h[i] = heuristic_task_2(coords[i][0], coords[goalPoint][0],
58                         coords[i][1], coords[goalPoint][1], mode)
59                     ;
60                     ncn.push(g[i] + h[i], h[i], i);
61                 }
62             }
63         };
64         if (parents[goalPoint] == -1)
65             return nullptr;
66         int i = goalPoint;
67         while (parents[i] != -1) {
68             path.push_back(parents[i]);
69             i = parents[i];
70         };
71         reverse(path.begin(), path.end());
72         path.push_back(goalPoint);
73         return build_linked_list(path, g, h, names);
74     };

```

### 3 Task 3 – Warehouse Robot Navigation with Obstacles

#### 3.1 Mô tả bài toán

Cho một nhà kho dạng lưới  $m \times n$  ( $0 = \text{ô trống}$ ,  $1 = \text{tường/chướng ngại vật}$ ). Robot di chuyển được theo 8 hướng:

- 4 hướng thẳng (Up, Down, Left, Right): chi phí = 1.0
- 4 hướng chéo (Up-Left, Up-Right, Down-Left, Down-Right): chi phí = 1.5

**Input:** Ma trận lưới, kích thước  $m \times n$ , tọa độ start, tọa độ goal, mode heuristic.

**Output:** Linked list PathNode chứa các bước di chuyển từ start đến goal. Mỗi node lưu tên hướng (ví dụ "Up-Right"),  $f(n)$ ,  $g(n)$ ,  $h(n)$ .

**Hàm chi phí:**

- $g(n)$ : tổng chi phí di chuyển thực tế từ start đến ô hiện tại
- $h(n)$ : ước lượng chi phí còn lại theo mode được chọn
- Mode 1 – Manhattan:  $h = |x_1 - x_2| + |y_1 - y_2|$
- Mode 2 – Chebyshev:  $h = \max(|x_1 - x_2|, |y_1 - y_2|)$

#### 3.2 Vì sao Manhattan không admissible ở bài này?

Xét 1 bước đi chéo từ  $(0, 0)$  đến  $(1, 1)$ :

- Chi phí thực tế: 1.5
- Manhattan:  $|1| + |1| = 2.0 > 1.5 \Rightarrow \text{không admissible}$
- Chebyshev:  $\max(1, 1) = 1.0 \leq 1.5 \Rightarrow \text{admissible}$

Do Manhattan đánh giá quá cao chi phí đi chéo, A\* Mode 1 sẽ **không đảm bảo tìm ra đường tối ưu** trong một số trường hợp ví dụ như test case 9 của task 3.

#### 3.3 Manual Run test case 9 task 3

Lưới 9x10, Start=(5,7), Goal=(0,3):

```
Col: 0 1 2 3 4 5 6 7 8 9
R0:  0 0 0 0 1 0 0 0 1 0    <- Goal (0,3)
R1:  0 1 1 1 1 0 0 1 0 0
R2:  0 0 0 0 0 0 0 0 0 0
R3:  0 1 1 1 0 1 0 0 0 1
R4:  0 1 1 0 1 0 1 1 0 0
R5:  0 1 1 0 0 0 1 0 0 0    <- Start (5,7)
R6:  0 1 0 1 1 0 0 0 1 1
R7:  0 0 0 1 1 0 0 0 0 1
R8:  0 0 0 0 0 0 0 0 0 0
```

**Mode 1 (Manhattan) – 6 bước đầu:**

Khởi tạo:  $g(5, 7) = 0$ ,  $h = |5 - 0| + |7 - 3| = 9$ ,  $f = 9$ . Push  $(5, 7)$ .

| Bước | Pop            | $g$ | $h$ | $f$  | Ghi chú                                               |
|------|----------------|-----|-----|------|-------------------------------------------------------|
| 1    | (5,7)          | 0   | 9   | 9    | Push Up-Right(4,8) và Down-Left(6,6), cùng $f = 10.5$ |
| 2    | (4,8) Up-Right | 1.5 | 9   | 10.5 | Push trước nên pop trước; push (3,7) $f = 10$         |
| 3    | (3,7) Up-Left  | 3.0 | 7   | 10   | Push (2,6) $f = 9.5$                                  |
| 4    | (2,6) Up-Left  | 4.5 | 5   | 9.5  | Push (1,5) $f = 9$                                    |
| 5    | (1,5) Up-Left  | 6.0 | 3   | 9    | Push (0,5) $f = 9$                                    |
| 6    | (0,5) Up       | 7.0 | 2   | 9    | Không có neighbor hợp lệ nào rẻ hơn                   |

Table 4: 6 bước đầu của A\* Mode 1 trên lưới 9x10

Thuật toán tiếp tục thêm 41 bước, đến đích ở bước 47 với  $g = 14.5$ .

#### Output Mode 1:

|                |         |         |      |
|----------------|---------|---------|------|
| Node: Up-Right | f: 10.5 | g: 1.5  | h: 9 |
| Node: Up-Left  | f: 10   | g: 3    | h: 7 |
| Node: Up-Left  | f: 9.5  | g: 4.5  | h: 5 |
| Node: Left     | f: 9.5  | g: 5.5  | h: 4 |
| Node: Left     | f: 9.5  | g: 6.5  | h: 3 |
| Node: Left     | f: 9.5  | g: 7.5  | h: 2 |
| Node: Left     | f: 11.5 | g: 8.5  | h: 3 |
| Node: Left     | f: 13.5 | g: 9.5  | h: 4 |
| Node: Up-Left  | f: 15   | g: 11   | h: 4 |
| Node: Up-Right | f: 14.5 | g: 12.5 | h: 2 |
| Node: Right    | f: 14.5 | g: 13.5 | h: 1 |
| Node: Right    | f: 14.5 | g: 14.5 | h: 0 |

Tổng chi phí  $g = 14.5$  – không tối ưu, do Manhattan overestimate.

#### Output Mode 2 (Chebyshev):

|                 |         |         |      |
|-----------------|---------|---------|------|
| Node: Down-Left | f: 7.5  | g: 1.5  | h: 6 |
| Node: Up-Left   | f: 8    | g: 3    | h: 5 |
| Node: Up        | f: 8    | g: 4    | h: 4 |
| Node: Up-Left   | f: 8.5  | g: 5.5  | h: 3 |
| Node: Up-Left   | f: 9    | g: 7    | h: 2 |
| Node: Left      | f: 10   | g: 8    | h: 2 |
| Node: Left      | f: 11   | g: 9    | h: 2 |
| Node: Up-Left   | f: 13.5 | g: 10.5 | h: 3 |
| Node: Up-Right  | f: 14   | g: 12   | h: 2 |
| Node: Right     | f: 14   | g: 13   | h: 1 |
| Node: Right     | f: 14   | g: 14   | h: 0 |

Tổng chi phí  $g = 14.0$  – tối ưu, Chebyshev admissible.

### 3.4 So sánh hai mode

| Tiêu chí                | Mode 1 (Manhattan)    | Mode 2 (Chebyshev)    |
|-------------------------|-----------------------|-----------------------|
| Công thức               | $ dx  +  dy $         | $\max( dx ,  dy )$    |
| Admissible với đi chéo? | Không ( $2.0 > 1.5$ ) | Có ( $1.0 \leq 1.5$ ) |
| Đảm bảo tối ưu?         | Không                 | Có                    |
| Chi phí tìm được        | 14.5                  | 14.0                  |
| Bước đầu từ Start       | Up-Right              | Down-Left             |

Table 5: So sánh Mode 1 và Mode 2, Task 3

### 3.5 Code hiện thực

Listing 4: Hàm heuristic\_task\_3\_4 và findWarehousePath

```

1 double heuristic_task_3_4(int x1, int x2, int y1, int y2, int mode) {
2     if (mode == 1) return abs(x1 - x2) + abs(y1 - y2);
3     if (mode == 2) return max(abs(x1 - x2), abs(y1 - y2));
4     return 0.0;
5 }
6
7 PathNode *findWarehousePath(int warehouse[100][100], int m, int n,
8                             int startX, int startY, int goalX, int goalY,
9                             int mode) {
10     double g[10000], h[10000];

```

```

10  bool visited[10000];
11  MinHeap ncn;
12  string names[10000];
13  int parents[10000];
14  vector<int> path;
15  const int dx[] = {-1, 1, 0, 0, -1, -1, 1, 1};
16  const int dy[] = {0, 0, -1, 1, -1, 1, -1, 1};
17  const double m_cost[] = {1.0, 1.0, 1.0, 1.0, 1.5, 1.5, 1.5, 1.5};
18  const string directions[] = {"Up", "Down", "Left", "Right",
19                               "Up-Left", "Up-Right", "Down-Left", "Down-Right",
20                               ""};
21
22  int startPoint = startX * n + startY;
23  int goalPoint = goalX * n + goalY;
24
25  if (startPoint == goalPoint) {
26      string name = "(" + to_string(startX) + "," + to_string(startY) + ")";
27      return new PathNode(name, 0, 0, 0);
28  }
29
30  for (int i = 0; i < m * n; i++) {
31      g[i] = MAX; h[i] = MAX;
32      visited[i] = false; parents[i] = -1;
33  }
34  g[startPoint] = 0;
35  h[startPoint] = heuristic_task_3_4(startX, goalX, startY, goalY, mode);
36  names[startPoint] = "(" + to_string(startX) + "," + to_string(startY) + ")";
37  ncn.push(g[startPoint] + h[startPoint], h[startPoint], startPoint);
38
39  while (!ncn.empty()) {
40      HeapNode temp = ncn.pop();
41      if (temp.id == goalPoint) break;
42      if (visited[temp.id]) continue;
43      visited[temp.id] = true;
44      int tx = temp.id / n, ty = temp.id % n;
45      for (int i = 0; i < 8; i++) {
46          int nx = tx + dx[i], ny = ty + dy[i];
47          if (nx >= 0 && nx < m && ny >= 0 && ny < n && warehouse[nx][ny] == 0)
48          {
49              int new_id = nx * n + ny;
50              double g_new = g[temp.id] + m_cost[i];
51              if (!visited[new_id] && g_new < g[new_id]) {
52                  g[new_id] = g_new;
53                  parents[new_id] = temp.id;
54                  names[new_id] = directions[i];
55                  h[new_id] = heuristic_task_3_4(nx, goalX, ny, goalY, mode);
56                  ncn.push(g[new_id] + h[new_id], h[new_id], new_id);
57              }
58          }
59      }
60
61      if (parents[goalPoint] == -1) return nullptr;
62      int i = goalPoint;
63      while (parents[i] != -1) { path.push_back(parents[i]); i = parents[i]; }
64      reverse(path.begin(), path.end());
65      path.push_back(goalPoint);
66      PathNode* trick = build_linked_list(path, g, h, names);
67      PathNode* head = trick->next;
68      delete trick;
69      return head;

```



## 4 Task 4 – Evacuation Route Planning

### 4.1 Mô tả bài toán

Cho một mặt bằng tòa nhà dạng lưới  $m \times n$  ( $0 = \text{ô trống}$ ,  $1 = \text{tường}$ ). Tìm đường sơ tán ngắn nhất từ vị trí (`startX`, `startY`) đến lối thoát (`exitX`, `exitY`).

**Input:** Ma trận lưới `floorPlan`, kích thước  $m \times n$ , tọa độ `start`, tọa độ `exit`, ma trận `weightMatrix[100][100]` (truyền vào để ghi kết quả), mode heuristic.

**Output:**

- Linked list `PathNode` chứa đường đi với tên node dạng "`(x, y)`",  $f(n)$ ,  $g(n)$ ,  $h(n)$
- `weightMatrix`: được điền như một **adjacency matrix** biểu diễn toàn bộ đồ thị của tòa nhà

**Hàm chi phí:** Giống Task 3 (thẳng = 1.0, chéo = 1.5; Mode 1 Manhattan, Mode 2 Chebyshev).

### 4.2 Chuyển đổi từ Grid sang Graph (Adjacency Matrix)

Đây là điểm khác biệt chính của Task 4 so với Task 3. Bài toán yêu cầu chuyển toàn bộ lưới sang dạng đồ thị và lưu vào `weightMatrix`.

**Quy tắc chuyển đổi:**

- Mỗi ô  $(x, y)$  trong lưới được ánh xạ thành node với ID  $u = x \cdot n + y$
- `weightMatrix[u][v]` = chi phí cạnh từ node  $u$  sang  $v$ : 1.0 (thẳng), 1.5 (chéo), 0.0 (không liền kề hoặc tường)
- Kích thước thực tế dùng:  $(m \cdot n) \times (m \cdot n)$

Listing 5: Build adjacency matrix trong `findEvacuationPath`

```
1 int total = m * n;
2 for (int i = 0; i < total; i++)
3     for (int j = 0; j < total; j++)
4         weightMatrix[i][j] = 0.0;
5
6 for (int x = 0; x < m; x++) {
7     for (int y = 0; y < n; y++) {
8         if (floorPlan[x][y] == 1) continue;
9         int u = x * n + y;
10        for (int i = 0; i < 8; i++) {
11            int nx = x + dx[i], ny = y + dy[i];
12            if (nx >= 0 && nx < m && ny >= 0 && ny < n && floorPlan[nx][ny]
13                == 0) {
14                int v = nx * n + ny;
15                weightMatrix[u][v] = m_cost[i];
16            }
17        }
18    }
```

### 4.3 Chạy tay ví dụ so sánh hai mode

**FloorPlan 3x3, tường tại (1,1), Start=(0,0), Exit=(2,2):**

```
Col: 0  1  2
Row 0: [ 0  0  0 ] <- Start (0,0), ID=0
Row 1: [ 0  W  0 ] <- tường tại (1,1), ID=4 không có cạnh
Row 2: [ 0  0  0 ] <- Exit (2,2), ID=8
```

**Bước 1: Ánh xạ ID ( $n = 3$ ):** Ô  $(x, y)$  có ID =  $x \cdot 3 + y$ . Ví dụ:  $(0, 0) = 0$ ,  $(0, 1) = 1$ ,  $(1, 0) = 3$ ,  $(1, 1) = 4$  (tường),  $(2, 2) = 8$ .

Một số cạnh tiêu biểu trong **weightMatrix** được build:

- $[0][1] = 1.0 - (0, 0) \rightarrow (0, 1)$  thẳng
- $[0][3] = 1.0 - (0, 0) \rightarrow (1, 0)$  thẳng
- $[0][4] = 0.0 - (1, 1)$  là tường, không có cạnh
- $[1][5] = 1.5 - (0, 1) \rightarrow (1, 2)$  chéo
- $[5][8] = 1.0 - (1, 2) \rightarrow (2, 2)$  thẳng xuống

**Bước 2: Chạy A\* so sánh Mode 1 và Mode 2:**

— **Mode 1 (Manhattan)** —

Công thức:  $h = |x_1 - x_2| + |y_1 - y_2|$ .  $h(\text{start}) = 4$ ,  $f_0 = 4$ .

| Bước | Pop           | $g$ | $h$ | $f$ | Hành động                                                         |
|------|---------------|-----|-----|-----|-------------------------------------------------------------------|
| 1    | Node 0 (0, 0) | 0.0 | 4   | 4.0 | Push 1 ( $g=1, h=3, f=4$ ), Push 3 ( $g=1, h=3, f=4$ )            |
| 2    | Node 1 (0, 1) | 1.0 | 3   | 4.0 | Push 2 ( $g=2, h=2, f=4$ ), Push 5 ( $g=2.5, h=1, f=3.5$ )        |
| 3    | Node 5 (1, 2) | 2.5 | 1   | 3.5 | <b>f nhỏ nhất nên pop trước 3.</b> Push 8 ( $g=3.5, h=0, f=3.5$ ) |
| 4    | Node 8 (2, 2) | 3.5 | 0   | 3.5 | <b>Goal!</b> Dừng. (Pop 4 node)                                   |

Table 6: A\* trace Task 4, Mode 1 Manhattan

— **Mode 2 (Chebyshev)** —

Công thức:  $h = \max(|x_1 - x_2|, |y_1 - y_2|)$ .  $h(\text{start}) = 2$ ,  $f_0 = 2$ .

| Bước | Pop           | $g$ | $h$ | $f$ | Hành động                                                               |
|------|---------------|-----|-----|-----|-------------------------------------------------------------------------|
| 1    | Node 0 (0, 0) | 0.0 | 2   | 2.0 | Push 1 ( $g=1, h=1, f=2$ ), Push 3 ( $g=1, h=2, f=3$ )                  |
| 2    | Node 1 (0, 1) | 1.0 | 1   | 2.0 | Push 2 ( $g=2, h=2, f=4$ ), Push 5 ( $g=2.5, h=1, f=3.5$ )              |
| 3    | Node 3 (1, 0) | 1.0 | 2   | 3.0 | <b>f của 3 (3.0) nhỏ hơn 5 (3.5) nên pop trước 5.</b> Push 6, Push 7... |
| 4    | Node 5 (1, 2) | 2.5 | 1   | 3.5 | Push 8 ( $g=3.5, h=0, f=3.5$ )                                          |
| 5    | Node 8 (2, 2) | 3.5 | 0   | 3.5 | <b>Goal!</b> Dừng. (Pop 5 node)                                         |

Table 7: A\* trace Task 4, Mode 2 Chebyshev

**Nhận xét từ 2 trace:** Cả hai mode đều tìm ra cùng đường đi  $(0, 0) \rightarrow (0, 1) \rightarrow (1, 2) \rightarrow (2, 2)$  với chi phí 3.5. Tuy nhiên:

- **Mode 1 (Manhattan)** đánh giá quá cao (overestimate) chi phí đi chéo ( $2.0 > 1.5$ ). Điều này vô tình làm cho  $f(5)$  giảm mạnh so với  $f(3)$ , khiến thuật toán lao thẳng về đích (chỉ pop 4 node). Nhưng trong các grid phức tạp khác, Mode 1 có thể không tìm được đường tối ưu.
- **Mode 2 (Chebyshev)** đánh giá đúng hoặc thấp hơn (admissible), khiến thuật toán cần thận duyệt qua cả nhánh node 3 (pop 5 node) trước khi chốt đường tối ưu đi qua node 5. Mode 2 luôn đảm bảo tối ưu.

**Output (chung cho cả 2 mode ở test case này):**

|       |        |        |        |        |
|-------|--------|--------|--------|--------|
| Node: | (0, 0) | f: 2/4 | g: 0   | h: 2/4 |
| Node: | (0, 1) | f: 2/4 | g: 1   | h: 1/3 |
| Node: | (1, 2) | f: 3.5 | g: 2.5 | h: 1/1 |
| Node: | (2, 2) | f: 3.5 | g: 3.5 | h: 0   |

#### 4.4 So sánh hai mode

| Tiêu chí                | Mode 1 (Manhattan)    | Mode 2 (Chebyshev)    |
|-------------------------|-----------------------|-----------------------|
| Công thức $h$           | $ dx  +  dy $         | $\max( dx ,  dy )$    |
| Admissible với đi chéo? | Không ( $2.0 > 1.5$ ) | Có ( $1.0 \leq 1.5$ ) |
| Đảm bảo tối ưu?         | Không                 | Có                    |

Table 8: Tổng kết so sánh Mode 1 và Mode 2, Task 4

#### 4.5 Code hiện thực

Listing 6: Hàm findEvacuationPath trong Algo.cpp

```

1 PathNode *findEvacuationPath(int floorPlan[100][100], int m, int n, int
  startX,
2                               int startY, int exitX, int exitY,
3                               double weightMatrix[100][100], int mode) {
4     double g[10000], h[10000];
5     bool visited[10000];
6     MinHeap mcn;
7     string names[10000];
8     int parents[10000];
9     vector<int> path;
10    const int dx[] = {-1, 1, 0, 0, -1, -1, 1, 1};
11    const int dy[] = {0, 0, -1, 1, -1, 1, -1, 1};
12    const double m_cost[] = {1.0, 1.0, 1.0, 1.0, 1.5, 1.5, 1.5, 1.5};
13
14    // Bước 1: Build adjacency matrix
15    int total = m * n;
16    for (int i = 0; i < total; i++)
17        for (int j = 0; j < total; j++)
18            weightMatrix[i][j] = 0.0;
19    for (int x = 0; x < m; x++) {
20        for (int y = 0; y < n; y++) {
21            if (floorPlan[x][y] == 1) continue;
22            int u = x * n + y;
23            for (int i = 0; i < 8; i++) {
24                int nx = x + dx[i], ny = y + dy[i];
25                if (nx >= 0 && nx < m && ny >= 0 && ny < n
26                    && floorPlan[nx][ny] == 0) {
27                    weightMatrix[u][nx * n + ny] = m_cost[i];
28                }
29            }
30        }
31    }
32
33    int startPoint = startX * n + startY;
34    int goalPoint = exitX * n + exitY;
35
36    if (startPoint == goalPoint) {
37        string name = "(" + to_string(startX) + ", " + to_string(startY) +
        ")";

```

```

38         return new PathNode(name, 0, 0, 0);
39     }
40
41     // Buoc 2: Khoi tao A*
42     for (int i = 0; i < m * n; i++) {
43         g[i] = MAX; h[i] = MAX;
44         visited[i] = false; parents[i] = -1;
45         names[i] = "(" + to_string(i/n) + ", " + to_string(i%n) + ")";
46     }
47     g[startPoint] = 0;
48     h[startPoint] = heuristic_task_3_4(startX, exitX, startY, exitY, mode);
49     ncn.push(g[startPoint] + h[startPoint], h[startPoint], startPoint);
50
51     // Buoc 3: Chay A* (dung weightMatrix tra chi phi canh)
52     while (!ncn.empty()) {
53         HeapNode temp = ncn.pop();
54         if (temp.id == goalPoint) break;
55         if (visited[temp.id]) continue;
56         visited[temp.id] = true;
57         int u = temp.id, tx = u / n, ty = u % n;
58         for (int i = 0; i < 8; i++) {
59             int nx = tx + dx[i], ny = ty + dy[i];
60             if (nx >= 0 && nx < m && ny >= 0 && ny < n) {
61                 int v = nx * n + ny;
62                 if (weightMatrix[u][v] != 0.0) {
63                     double g_new = g[u] + weightMatrix[u][v];
64                     if (!visited[v] && g_new < g[v]) {
65                         g[v] = g_new; parents[v] = u;
66                         h[v] = heuristic_task_3_4(nx, exitX, ny, exitY,
67                                                 mode);
68                         ncn.push(g[v] + h[v], h[v], v);
69                     }
70                 }
71             }
72         }
73         if (parents[goalPoint] == -1) return nullptr;
74
75         // Buoc 4: Truy vet va tra ve linked list
76         int i = goalPoint;
77         while (parents[i] != -1) { path.push_back(parents[i]); i = parents[i]; }
78         reverse(path.begin(), path.end());
79         path.push_back(goalPoint);
80         return build_linked_list(path, g, h, names);
81     }

```